BT4012 Competition [2026] report: Detecting Illicit Bitcoin Transactions Using Ensemble Models and Graph Features

Student ID: E1122390
Student Name: Marvin Choo Ming Han

GitHub repo: https://github.com/marvinchoo-mh/BT4012-Kaggle-Competition-e1122390

1. Introduction

Bitcoin transactions are publicly recorded, but the identities of those involved are often unknown. This makes Bitcoin attractive for illicit activities such as money laundering and scams. Detecting suspicious transactions is therefore important for fraud prevention and financial security.

The BT4012 Kaggle Competition aims to distinguish licit (legitimate) from illicit (suspicious) Bitcoin transactions. The dataset contains 157,101 transactions, each with 165 anonymised features and connections to other transactions. I identified a few key challenges, including the large number of unlabelled transactions, class imbalance, and changing transaction patterns over time.

During my exploratory data analysis, I observed data drift, where certain feature values changed significantly over time. This raised concerns that models trained on historical transactions might perform poorly on future data. To address this, I converted drifting but useful features into percentile ranks and removed less reliable features. I also observed that illicit transactions tend to be connected to other illicit transactions. Based on this, I used neighbour-score stacking to incorporate the predicted risk scores of connected transactions, rather than evaluating each transaction in isolation. To evaluate performance on future data, I used time based validation, training on earlier time steps and testing on later ones rather than randomly splitting transactions. My final model was an ensemble of 11 models using different algorithms and feature sets to improve prediction reliability.

2. Methodology

2.1. Data Preprocessing and Feature Engineering

Data preparation and class imbalance

The training data contains 141,772 transactions, but only 31,235 have known labels: 3,644 illicit (11.7%) and 27,591 licit. The remaining 110,537 transactions are unlabelled. I retained these transactions because their features and network connections still provide useful information. As the features were already normalised and contained no missing values, additional scaling or imputation was unnecessary.

Handling data drift

Some features changed substantially over time. For example, `feat_101` was strongly correlated with time step (0.994), meaning a model might learn when a transaction occurred rather than whether it was illicit. To reduce this risk, I calculated each feature's percentile rank within its time step, so a transaction could be compared with others from the same period. I then used LightGBM-based scores to assess (1) how much a feature changed over time and (2) how useful it was for predicting illicit activity. Highly drifting but useful features were converted to ranks; highly drifting and less useful features were removed; the others were kept in their original form. This retained 142 raw features, converted 19 to ranks, and removed 11. I ranked the retained features by predictive importance for later model comparisons.

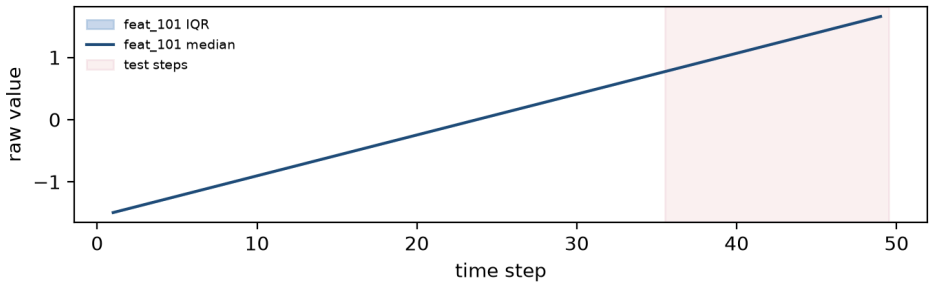


Figure 1: Drift in `feat_101` across time steps (median and inter-quartile range); test steps 36–49 shaded.

Graph based feature engineering

I also explored graph based feature engineering to capture relationships between Bitcoin transactions. I constructed a transaction graph for each time step and extracted seven features: in-degree, out-degree and total degree (number of connections), PageRank (importance within the network), clustering coefficient (how closely neighbours are connected), connected-component size (size of the connected group), and k-core number (how strongly connected a transaction's surrounding group is). My analysis showed that 71.0% of illicit transactions had exactly one incoming connection, compared to 45.4% of licit transactions, suggesting that transaction connections could help identify illicit activity. However, none of these graph features ranked among the top 44 selected features. Instead, I decided to make better use of transaction relationships through neighbour-score stacking, which incorporates the predicted risk scores of connected transactions (Section 2.3).

2.2. Model Training and Validation

I trained several tree based models, focusing on evaluating predictions fairly, handling class imbalance, and reducing overfitting.

Time based validation

Since the competition's test data covers 14 future time steps (36–49), I used a similar approach for validation. I trained my models on steps 1–21 and evaluated them on the following 14 steps (22–35), rather than randomly splitting transactions. Earlier experiments using shorter validation periods produced ROC-AUC scores above 0.99, but these were overly optimistic because transaction patterns may change over longer periods. After selecting my final approach, I retrained the models on all available training steps (1–35).

Model training

I experimented with LightGBM and XGBoost (gradient-boosted trees), as well as Random Forest and ExtraTrees. I used 800 trees and a learning rate of 0.02 for the boosting models, and 400 trees each for the forest models. I selected these algorithms because they learn different patterns from the data and may make different prediction errors.

Handling class imbalance

Since only 11.7% of labelled training transactions were illicit, I assigned higher training weights to illicit transactions to help the models learn from this minority class. I also experimented with including unlabelled transactions as weak examples of legitimate activity, assigning them small weights (0.05 or 0.15) because some could actually be illicit.

Reducing overfitting

To reduce overfitting, I limited the number of selected features and averaged predictions from the main ensemble members across three random seeds. I also used out-of-fold predictions for neighbour-score stacking, meaning the risk scores were generated by models that had not trained on those transactions. This helped prevent information from the training labels from leaking into the stacking features.

2.3. Model Comparison and Final Ensemble

I compared different feature sets and models using time based validation to identify an approach that performed well despite changes in transaction patterns.

Neighbour-score stacking

Since illicit transactions tend to be connected to other illicit transactions, I let the model learn from its neighbours' predicted risk scores in three stages:

1. Initial prediction: Predict each transaction's illicit risk from its own features.
2. Neighbour information: Calculate the average and maximum predicted risk among incoming, outgoing and all connected transactions, and use these as additional features for new predictions.
3. Refined prediction: Repeat using the updated neighbour scores. This allows information from transactions beyond a node's direct neighbours to influence its prediction indirectly.

Feature and model comparison

I evaluated three feature counts (30, 44 and 60) with LightGBM and XGBoost using three stacking stages. Table 1 gives validation ROC-AUC.

Model	30 features	44 features	60 features
LightGBM	0.8631	0.8629	0.8690
XGBoost	0.8516	**0.8858**	0.8728

Table 1: Validation ROC-AUC by algorithm and feature count (train steps 1–21, validate steps 22–35).

Although XGBoost scored highest with 44 features, its performance varied more across feature counts, whereas LightGBM was more consistent. I therefore preferred LightGBM as the main model and retained 44 features as a compromise between performance and stability.

Final ensemble

My final model was an ensemble of 11 models, combining LightGBM, ExtraTrees and Random Forest with different feature sets and treatments of unlabelled transactions. I chose this because no single model was consistently strongest across the feature sets tested. Combining them may capture different patterns and reduce reliance on any one model as the data changes.

I combined the models' prediction rankings using a weighted average. Three models were first averaged into a core, which was then combined with eight other models. The core and other top-44-focused models accounted for 7 of the 12 final weight units; other feature views received smaller weights to add complementary information. I chose fixed weights manually, rather than optimising them on a single validation split, to reduce overfitting. The final ensemble was retrained on steps 1–35 for the test predictions.

3. Results and Discussion

The final ensemble achieved a public leaderboard ROC-AUC of 0.96974. I also evaluated it on the held out time based validation period (steps 22–35).

Evaluation	ROC-AUC	Precision	Recall	F1
Time-based validation	0.8483	0.9911	0.6272	0.7682
Public leaderboard	0.96974	N/A	N/A	N/A

Table 2: Final ensemble performance. Precision, recall and F1 use the highest-scoring 9.6% of labelled validation transactions; the leaderboard reveals ROC-AUC only.

Transactions flagged	Precision	Recall	F1
Top 5%	1.0000	0.3305	0.4968
Top 9.6% (main)	**0.9911**	**0.6272**	**0.7682**
Top 10%	0.9846	0.6503	0.7833
Top 15%	0.7483	0.7411	0.7447

Table 3: Precision, recall and F1 of the final ensemble on the validation steps when the top-scored transactions are flagged as illicit; ROC-AUC (0.8483) does not depend on the cut-off.

Performance interpretation

I flagged the top 9.6% of validation transactions because 1,871 of the 19,530 labelled transactions in training steps 1–21 (9.6%) were illicit. This cutoff was chosen from the training period, not tuned to the validation labels. Precision was 99.1%: nearly all flagged cases were truly illicit. Recall was 62.7%: approximately 37% of illicit transactions were missed. Table 3 shows this trade-off: flagging only the top 5% gives 100% precision but 33% recall, while flagging 15% raises recall to 74% and lowers precision to 75%. This illustrates a practical trade-off: flagging fewer cases reduces false alarms and investigation workload, but also risks missing more illicit activity. My model could prioritise high confidence cases, with a higher recall approach or lower alert threshold used to identify more cases for human review. The 9.6% cutoff affects precision and recall, but not ROC-AUC.

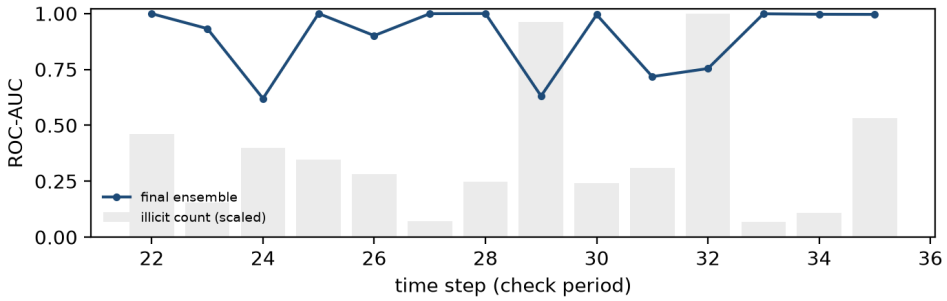


Figure 2: Per-step validation ROC-AUC of the final ensemble (steps 22–35); grey bars show the number of illicit transactions (scaled).

Performance across time periods

The model performed well in most validation periods but struggled in steps 24, 29, 31 and 32 (Figure 2). This suggests that changes in transaction patterns may still affect its performance despite my efforts to handle data drift. The validation and leaderboard scores also differ because they were evaluated on different time periods, with the final model trained on more data.

Strengths and limitations

A key strength of my approach was using time based validation and feature selection to account for changing transaction patterns. Neighbour-score stacking also allowed me to learn from connected transactions. However, treating unlabelled transactions as likely legitimate may introduce errors. Additionally, I manually selected the ensemble weights and relied on only one time based validation split, which may limit how well my model performs on future data.

4. Conclusion and Reflection

My final ensemble achieved a public leaderboard ROC-AUC of 0.96974. The project showed me how graph relationships can reveal patterns that individual transaction features miss, and why time must be considered when developing machine learning models. Neighbour-score stacking used connected transactions to improve the information available for predictions, while time based validation revealed weaknesses that shorter checks did not show.

Given more time, I would explore community detection to identify suspicious clusters and graph neural networks to learn more complex transaction relationships. I would also use multiple time based validation splits and investigate better approaches to unlabelled data. For real world use, I believe my high precision model could be complemented by a higher recall model that flags a wider range of suspicious transactions for human review. Regular monitoring and retraining would help address changes in transaction behaviour.